

CruiseBench: A Real-Flight-Aligned N-CMAPSS Benchmark for Engine RUL Prediction

Pu Cheng
cheng.chengpu123@gmail.com

Qiang Miao
mqiang@scu.edu.cn

July 23, 2026

Abstract

Remaining useful life (RUL) prediction estimates how long an engine can continue safe operation and is central to maintenance planning. N-CMAPSS extends C-MAPSS by simulating run-to-failure aero-engine trajectories using recorded real-flight profiles and retaining complete within-flight time series rather than cycle-level snapshots. However, this added realism reduces evaluation control because full-flight records increase data volume and entangle degradation cues with operating-regime variation, complicating preprocessing choices and direct comparisons of RUL modeling performance. To mitigate this issue, this paper proposes CruiseBench, a cruise-stage RUL benchmark derived from N-CMAPSS. It introduces CPM-N-CMAPSS (Cruising-Period Mask for N-CMAPSS), a mask artifact that stores cycle-local cruising intervals identified by the common-altitude method for the nine accessible subdatasets. CruiseBench applies a fixed protocol to the masked rows, using scenario descriptors and measured sensors as inputs while excluding virtual sensors, health parameters, and auxiliary metadata from the feature tensor, preserving native-resolution windows, and applying dataset-wise RUL caps. Experiments with LSTM, GRU, TCN, and TSMixer provide baseline results for this setting. Under CruiseBench-eta5-W256-S10, TSMixer obtains the lowest average RMSE, 3.46 ± 1.71 , and Saxena score, $(2.50 \pm 2.99) \times 10^4$. Ablation studies show that flight-stage selection, temporal downscaling method, and RUL-cap threshold affect reported results. With its fixed cruise-stage protocol, CruiseBench provides a reproducible sub-benchmark for controlled RUL model comparison and CPM-N-CMAPSS provides a stage-specific data foundation for future transfer-learning and domain-adaptation studies. The code is available at <https://github.com/NostalgiaJohn/CruiseBench>.

1 Introduction

Remaining useful life (RUL) prediction is a central task in prognostics and health management because maintenance planning depends on estimating how long monitored assets can continue safe operation. Aero-engines are a representative application. They are safety critical, heavily instrumented, and operated under changing environmental and control conditions. The original C-MAPSS benchmark provided simulated turbofan run-to-failure trajectories and became a standard testbed for data-driven prognostics [Saxena et al., 2008]. N-CMAPSS extends this line of work by generating run-to-failure trajectories under real recorded flight profiles and operation-history-dependent degradation [Arias Chao et al., 2021].

The realism of N-CMAPSS also creates a benchmark-design issue. A complete flight cycle contains climb, cruise, and descent, and these stages differ clearly in altitude, Mach number, throttle-resolver angle, temperature, and sensor response. Full-flight records also increase data volume and make preprocessing choices more influential. A model evaluated on full-cycle trajectories may therefore benefit from both degradation-sensitive temporal patterns extraction and operating-regime variation detection. Cruise-stage benchmarking protocol can isolate one stable operating regime for controlled RUL model comparison. It reduces variation caused by

mixing flight stages while preserving a real-flight-aligned prediction task, thereby improving comparability across RUL models.

This paper proposes CruiseBench, a cruise-stage benchmark derived from N-CMAPSS. It defines CPM-N-CMAPSS (Cruising-Period Mask for N-CMAPSS), a reproducible mask artifact that stores accepted cruise intervals inside each flight cycle while retaining the original N-CMAPSS data splits and organization. The resulting evaluation setting focuses on cruise-stage RUL prediction, and the stage-specific mask can also support later transfer-learning and domain-adaptation studies across operating regimes. The ablation studies in Section 6.2 support this design by showing that flight-stage selection, temporal downscaling method, and RUL-cap threshold affect reported results. Together, these findings motivate treating flight stage, temporal downscaling, and cap threshold as explicit benchmark settings.

The contributions are:

- CPM-N-CMAPSS, a mask artifact that stores reproducible cycle-local cruising intervals without modifying or redistributing the official raw files, with coverage across nine accessible subdatasets.
- CruiseBench, a fixed cruise-stage RUL protocol that specifies the input feature groups, meta-data exclusions, windowing rule, RUL-cap rule, and evaluation metrics.
- a stage-specific data foundation for future transfer-learning and domain-adaptation studies across operating regimes.

The rest of the paper is organized as follows. Section 2 reviews related RUL benchmarks and protocol-comparability issues. Section 3 describes the source data, mask artifact, and cruising-period extraction method. Section 4 defines the supervised RUL protocol, including feature selection, RUL capping, windowing, and metrics. Section 5 describes the baseline models and training setup. Section 6 reports the main results and ablations. Section 7 discusses limitations. Section 8 summarizes the paper and future directions. Section 9 documents reproducibility.

2 Related Work

2.1 C-MAPSS and N-CMAPSS benchmarks

C-MAPSS established a widely used simulated turbofan degradation benchmark for RUL prediction [Saxena et al., 2008]. It generates run-to-failure trajectories for a fleet of large turbofan engines by changing flow and efficiency parameters in the engine model. Each engine cycle is represented by one operating and sensor snapshot rather than by a complete within-flight time series. This representation greatly reduces data volume and makes algorithm comparison easier on C-MAPSS, but it does not fully describe flight-stage behavior inside a complete flight.

N-CMAPSS extends C-MAPSS by using real recorded flight profiles and operation-history-dependent degradation in a higher-fidelity simulation framework [Arias Chao et al., 2021]. It simulates complete flights with climb, cruise, and descent stages, and stores development and test data in HDF5 files. Each file provides scenario descriptors W , measured sensors X_s , virtual sensors X_v , health parameters θ , RUL labels, and auxiliary metadata such as unit, cycle, flight class, and health state. These additions make the benchmark more realistic, but they also introduce more choices in preprocessing and evaluation.

2.2 Protocol Variation in N-CMAPSS Studies

Reported N-CMAPSS results differ not only by model architecture, but also by protocol choices. Table 2.1 summarizes representative and latest choices reported in N-CMAPSS RUL studies. The table concludes subdataset coverage, temporal reduction, window stride, target capping, and feature groups in columns.

Table 2.1: Representative N-CMAPSS RUL protocol choices in recent work. The sample-rate factor is relative to the native N-CMAPSS time resolution. NR means not reported.

Study	Datasets used	Sample-rate factor	Window size / stride	RUL cap	Input channels or feature rule
Arias Chao et al. [2022]	DS02	x10	50 / 1	NR	$W + X_s$, hybrid adds $\hat{X}_s + \hat{X}_v + \theta$
Custode et al. [2022]	DS02	x100	NR	NR	20 condition-monitoring signals
Zeng and Liang [2023]	DS01–DS07, DS08a, DS08c	x100	30 / 1	No early cap	$W + X_s + X_v$
Xu et al. [2024]	DS01–DS07	NR	NR	NR	Zero-valued N-CMAPSS variables removed, retained group NR
Wang et al. [2024]	DS01–DS07	NR	40 / 1	65	$W + X_s$
Song et al. [2025]	DS02	x100	100, 200 / NR	NR	W for clustering, sensor channels for graph learning
Forest et al. [2025]	DS01, DS04, DS05, DS07	x10	NR	Flat or linear RUL option	Concept-bottleneck inputs with degradation-mode concepts
Wu et al. [2025]	DS02	x10	60 / 1	NR	$X_s + X_v + \theta$, seven constant θ columns removed
Zhang et al. [2025]	DS02	x1000	70 / 1	NR	$W + X_s + X_v$
Landau et al. [2026]	DS02	x20	50 / 10	NR	W + selected X_s
Qin et al. [2026]	DS02	x1	60 / 1	120	$W + X_s$
Ren et al. [2026]	DS01–DS07, DS08a	x100	2 / 1	NR	N-CMAPSS feature group NR
Zhu et al. [2026]	DS02	x10	NR, patch 16 / 8	NR	X_s
Gao et al. [2025]	Subset NR	NR	NR	NR	N-CMAPSS feature group NR
Zhou et al. [2026]	Subset NR	NR	NR	NR	N-CMAPSS feature group NR
CruiseBench (this work)	DS01–DS07, DS08a, DS08c	x1	256 / 10	Dataset-wise $\eta = 5\%$ caps	$W + X_s$

The input-feature construction is one important source of variation. The original N-CMAPSS data-driven setting uses scenario descriptors W and measured sensors X_s as inputs [Arias Chao et al., 2021]. Arias Chao et al. compare this data-driven rule with hybrid variants that add calibrated sensor estimates $\hat{X}_s$, virtual sensors $\hat{X}_v$, and health parameters θ [Arias Chao et al., 2022]. Other studies in Table 2.1 use combinations such as $W + X_s + X_v$, $X_s + X_v + \theta$, selected sensor subsets, or concept variables. These settings can be appropriate for their respective objectives, but inconsistent sensor selection complicates benchmark construction and direct comparison.

The N-CMAPSS subdataset choice is also part of the experimental setting. DS02 is one of the N-CMAPSS datasets used in prior prognostics studies [Arias Chao et al., 2021]. It contains nine run-to-failure units across flight classes 1, 2, and 3, includes complete flight traces and several variable groups, and is useful for studies beyond a simple data-driven RUL setting. However, DS02-only evaluation can narrow the benchmark scope. As Table 2.1 shows, several studies report DS02 only [Arias Chao et al., 2022, Landau et al., 2026, Qin et al., 2026, Wu et al., 2025, Zhang et al., 2025]. Other studies use selected groups, such as DS02, DS05, DS06, and DS07, or DS01–DS07 [Liu et al., 2024, Xu et al., 2024, Wang et al., 2024]. A nine-subdataset setting, namely DS01–DS07, DS08a, and DS08c, is also used after excluding DS08d [Zeng and Liang, 2023, Huang et al., 2025]. Ren et al. instead evaluate DS01–DS07 and DS08a [Ren et al., 2026]. These examples show that subdataset coverage is part of the reported setting rather than a fixed community default.

Preprocessing choices introduce another source of variation. Because N-CMAPSS contains millions of samples in each subset, studies may reduce the training and testing burden through temporal downscaling methods such as sample-rate reduction or window-stride settings. For

example, Liu et al. downsample the selected data for this reason [Liu et al., 2024]. Among the studies summarized in Table 2.1, reported sample-rate factors range from native resolution to x1000, while window lengths, strides, and patch sizes also vary or are left unreported. The table also shows that target construction is not uniform. RUL caps may be not used, fixed to a stated value, dataset-wise, or not reported. Reported results can further depend on whether health-state or flight-class metadata are used, the metric definition, and whether a number is selected from one run or averaged across repeated runs. These differences make direct comparison difficult, and results reported under different procedures should not be treated as one unified leaderboard.

The last row of Table 2.1 states that the proposed CruiseBench fixes a cruise-stage setting across the nine accessible subdatasets, native sample rate, 256-sample windows with stride 10, dataset-wise label caps, and model inputs restricted to the original data-driven groups $W + X_s$. Making these choices explicit helps distinguish gains from model design from gains caused by preprocessing, feature selection, or target construction.

3 Source Data and Mask Construction

CPM-N-CMAPSS (Cruising-Period Mask for N-CMAPSS) is a cruise-stage mask artifact for N-CMAPSS. N-CMAPSS contains complete run-to-failure aero-engine trajectories generated by using recorded flight profiles as inputs to the C-MAPSS turbofan model and by imposing degradation through component health parameters [Arias Chao et al., 2021]. In CPM-N-CMAPSS, each accepted cruising period is stored as interval metadata in the local sample coordinates of one unit-cycle trace. Each entry records the dataset split, unit, cycle, and start–end sample indices needed to select the corresponding official HDF5 rows. Benchmark construction applies this mask to obtain the cruise-stage subset while leaving the source files unchanged. This section describes the source-file structure, the mask artifact, and the cruising-period extraction method.

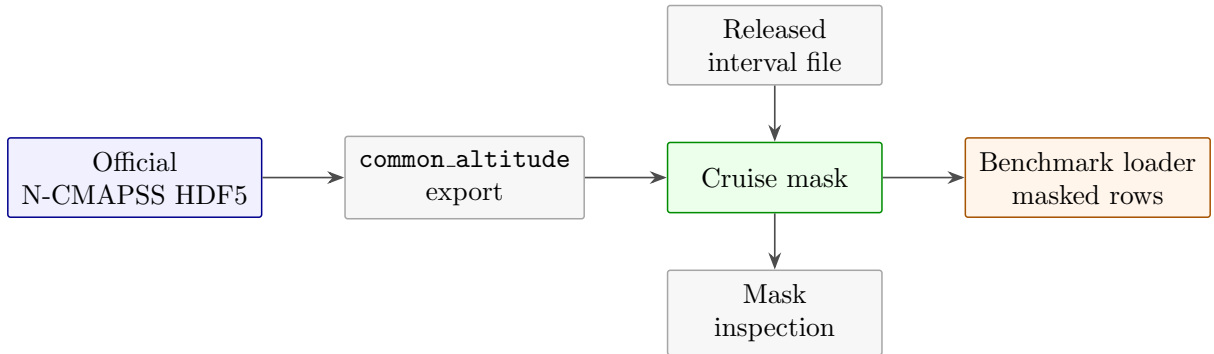


Figure 3.1: CPM-N-CMAPSS mask workflow. The mask can be reproduced from official N-CMAPSS HDF5 files or reused from the released interval file. Inspection tools check the same artifact before the benchmark loader applies it to source rows.

3.1 Mask Artifact

The HDF5 mask file stores interval metadata for accepted unit-cycle pairs. Each interval is expressed by local sample indices inside one flight cycle. This design keeps the artifact small, avoids duplicating N-CMAPSS sensor arrays, and makes the mask auditable with the original altitude trace.

3.2 Open-source Mask Workflow

The open-source package supports both mask reproduction and direct reuse of the released mask, as summarized in Figure 3.1. Users place official N-CMAPSS files in the local data directory, run the common-altitude export tool when reproduction is needed, and inspect the generated intervals. During training, the data loader applies the checked mask to the official source rows and then follows the benchmark rules defined in Section 4.

3.3 Source Files and Cruise Mask

The source data are the N-CMAPSS HDF5 files. Each dataset split provides scenario descriptors W , measured sensors X_s , virtual sensors X_v , auxiliary metadata A , health parameters θ , and RUL labels. The auxiliary metadata include unit id, cycle id, flight class, sample index, and health state. CPM-N-CMAPSS covers nine accessible subdatasets of N-CMAPSS: DS01-005, DS02-006, DS03-012, DS04, DS05, DS06, DS07, DS08a-009, and DS08c-008. DS08d-010 is excluded because its available HDF5 file could not be read. Table 3.1 summarizes the released mask file.

Table 3.1: CPM-N-CMAPSS cruise-mask export summary. Cycle ratio is accepted cycles divided by total cycles. Sample ratio is accepted cruise samples divided by total samples.

Dataset	Cycles	Accepted cycles	Cycle ratio	Sample ratio
DS01-005	894	835	0.934	0.264
DS02-006	648	611	0.943	0.340
DS03-012	1101	1026	0.932	0.259
DS04	856	827	0.966	0.252
DS05	818	768	0.939	0.273
DS06	797	747	0.937	0.273
DS07	812	762	0.938	0.268
DS08a-009	994	943	0.949	0.261
DS08c-008	553	534	0.966	0.267
Total	7473	7053	0.944	0.271

3.4 Cruising-Period Extraction

Cruising-period extraction assigns cycle-local sample indices to the part of a flight cycle selected as cruise. Let a cycle contain n samples indexed by $i = 0, 1, \dots, n-1$, and let a_i denote the raw altitude at sample i . The released mask uses a fixed common-altitude method, denoted by

$$\mathcal{C}_\theta : (a_0, \dots, a_{n-1}) \mapsto \mathcal{I}_c, \quad (1)$$

where θ denotes the fixed method parameters. The output $\mathcal{I}_c$ is either a singleton set $\{(s, e)\}$ with $0 \leq s \leq e < n$ or an empty set when no cruising period is accepted. The exported interval is expanded into a binary sample when loaded:

$$m_i = \begin{cases} 1, & \text{if } \mathcal{I}_c = \{(s, e)\} \text{ and } s \leq i \leq e, \\ 0, & \text{otherwise.} \end{cases} \quad (2)$$

3.4.1 Common-altitude method

The common-altitude method searches each cycle for a sustained high-altitude level segment. It first smooths altitude with a 31-sample centered rolling median:

$$\tilde{a}_i = \text{median}\{a_j : j \in \mathcal{W}_i\}, \quad (3)$$

where

$$\mathcal{W}_i = \{j : 0 \leq j < n, |j - i| \leq 15\}. \quad (4)$$

The cycle is processed only when its smoothed altitude span is large enough:

$$S = \max_i \tilde{a}_i - \min_i \tilde{a}_i. \quad (5)$$

If $S < 500$ ft, no cruise interval is exported for that cycle.

Next, the method defines a lower altitude floor:

$$F = \max \left(Q_{0.50}(\tilde{a}), \min_i \tilde{a}_i + 0.35S \right). \quad (6)$$

Here, $Q_{0.50}(\tilde{a})$ is the median of the smoothed altitude sequence. Only candidate altitude levels at or above F are considered.

The method then searches for repeated high-altitude levels using shifted bins. Let $\Delta = 50$ ft be the bin width and $K = 50$ be the number of bin shifts. For shift $k = 0, \dots, K - 1$, the bin origin is $o_k = k\Delta/K$, and sample i is assigned to the bin center

$$b_i^{(k)} = \left\lfloor \frac{\tilde{a}_i - o_k}{\Delta} \right\rfloor \Delta + o_k + \frac{\Delta}{2}. \quad (7)$$

For each shift, the method selects the three most frequent bin centers among samples with $b_i^{(k)} \geq F$. For a selected candidate level λ , the initial candidate mask is

$$r_i(\lambda, k) = \begin{cases} 1, & \text{if } |\tilde{a}_i - \lambda| \leq 50, \\ 0, & \text{otherwise.} \end{cases} \quad (8)$$

Short false gaps of at most 80 samples are merged when the adjacent smoothed altitude values still belong to the same plateau. The method then extracts continuous true runs from the repaired candidate mask. A run from s to e is accepted only if

$$e - s + 1 \geq 512 \quad \text{and} \quad \text{std}\{a_i : s \leq i \leq e\} \leq 50 \text{ ft}. \quad (9)$$

Among all accepted runs from all shifts and candidate levels, the mask export keeps the longest one and writes its endpoints in cycle-local sample indices. If no run satisfies these criteria, no cruise interval is exported for that cycle.

3.4.2 Extraction examples

Figure 3.2 shows representative accepted cruising periods from DS01-005. The selected interval is generated from altitude within each cycle and exported as cycle-local start and end sample indices.

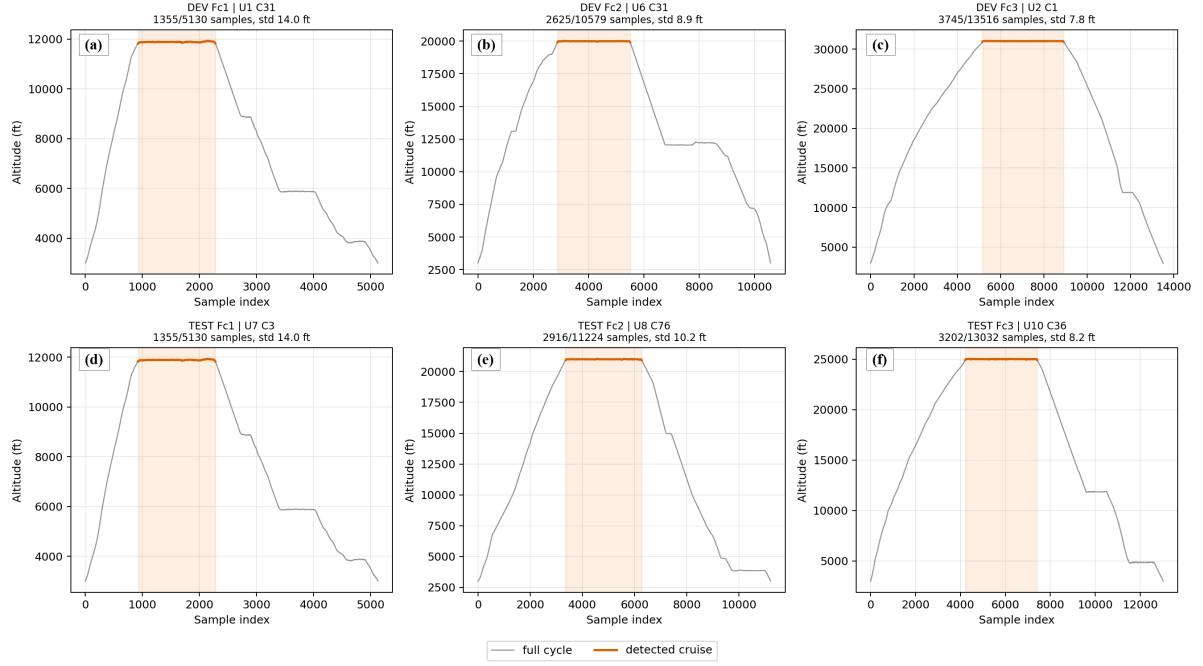


Figure 3.2: Accepted cruising-period examples from DS01-005. The mask is generated cycle by cycle from altitude and then exported as cycle-local start and end sample indices.

Cycles that do not contain a sustained common-altitude segment are rejected. Figure 3.3 shows two DS01-005 development examples: Unit 1 Cycle 6 has no sustained high-altitude plateau, while Unit 1 Cycle 11 contains a visually plausible level segment that lasts only 281 samples, below the 512-sample minimum. These rejected cycles receive no fallback interval and are omitted at the cycle-local matching step.

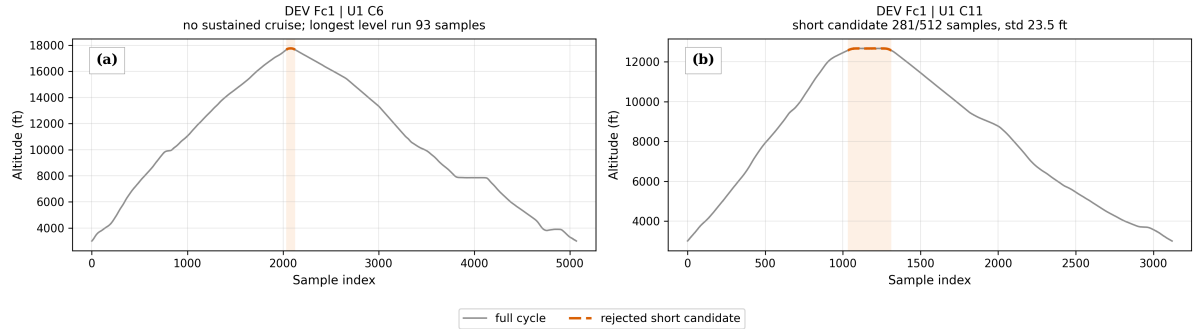


Figure 3.3: Examples of DS01-005 cycles skipped by the common-altitude method. Orange dashed segments show the longest candidate level run found before the minimum-duration test. (a) Unit 1 Cycle 6 does not form a sustained cruise-level plateau; its longest level run is 93 samples. (b) Unit 1 Cycle 11 contains a short stable candidate, but 281 samples is below the 512-sample selection threshold.

4 Benchmark Definition

4.1 Task Formulation

CruiseBench follows the N-CMAPSS prognostics problem definition introduced by Arias Chao et al. [Arias Chao et al., 2021]. Given a time window of operating and sensor measurements,

the model predicts the remaining useful life of the engine. CruiseBench keeps this supervised RUL regression formulation and applies it to the cruise-stage rows selected by the mask. The model input includes only the scenario descriptors W and measured sensors X_s . In the official N-CMAPSS files, W provides four channels and X_s provides fourteen channels. Let T denote the window length and let $d = 18$ denote the number of input channels. A windowed input therefore has shape

$$X_j \in \mathbb{R}^{T \times d}. \quad (10)$$

Virtual sensors X_v , health parameters θ , and auxiliary metadata A are not used as predictive features. Metadata fields such as unit id, cycle id, flight class, sample index, and health state are used only for filtering or offline setting design.

For a model f_ϕ , the benchmark prediction for window X_j is

$$\hat{y}_j = f_\phi(X_j). \quad (11)$$

4.2 RUL Target Construction

Cap is applied to the original N-CMAPSS RUL labels to form the supervised target. RUL capping is a common target-labeling step in C-MAPSS studies: the target is kept constant during the early-life part and then follows the true RUL near failure [Heimes, 2008, Li et al., 2018, Ellefsen et al., 2019]. This makes the training target emphasize the late-life region where degradation is clearer. Many C-MAPSS comparisons use a single cap, often 125 cycles, because the benchmark is organized around cycle-level trajectories with a relatively compact labeling rule.

N-CMAPSS requires a dataset-wise rule. It contains complete flight traces, multiple sub-datasets, and health-state metadata generated by the simulator. Health state is not used as a model input in CruiseBench, but it can be used offline to define dataset-wise target caps. Therefore caps are derived from the exponential abnormal-degradation law and treat the threshold η as part of the benchmark setting rather than as a directly observed detectability limit. Following the N-CMAPSS abnormal degradation model [Arias Chao et al., 2021], offset and noise are ignored and the abnormal damage magnitude is written as

$$D(t) = \exp(at^b) - 1, \quad (12)$$

with $a \in [0.001, 0.003]$ and $b \in [1.4, 1.6]$. For dataset i with minimum abnormal length L_i , define a normalized curve

$$\bar{D}_i(u) = \frac{\exp(aL_i^b u^b) - 1}{\exp(aL_i^b) - 1}, \quad u \in [0, 1]. \quad (13)$$

Given a detectable-damage threshold η , solve $\bar{D}_i(u_\eta) = \eta$ and set

$$\text{MAX_RUL}_i = \text{round}(L_i(1 - u_\eta)). \quad (14)$$

The regression target for a window from dataset i is therefore

$$y = \min(\text{RUL}, \text{MAX_RUL}_i). \quad (15)$$

This transformation keeps the late-life target unchanged while mapping earlier windows to a dataset-specific plateau. Table 4.1 lists the minimum abnormal length used for each dataset and the resulting caps for the main setting and the two sensitivity settings. The main setting uses $a = 0.002$, $b = 1.5$, and $\eta = 5\%$. The $\eta = 10\%$ setting uses a stricter cap, while the $\eta = 0\%$ setting uses L_i itself as the dataset-wise cap. The three settings are reported separately when used.

Table 4.1: Minimum abnormal lengths and dataset-wise RUL caps. The main setting uses the $\eta = 5\%$ cap.

Dataset	Minimum abnormal length L_i	$\eta = 0\%$ cap	$\eta = 5\%$ cap	$\eta = 10\%$ cap
DS01-005	46	46	38	34
DS02-006	41	41	34	31
DS03-012	41	41	34	31
DS04	52	52	43	38
DS05	46	46	38	34
DS06	50	50	41	37
DS07	47	47	39	35
DS08a-009	40	40	34	30
DS08c-008	33	33	28	25

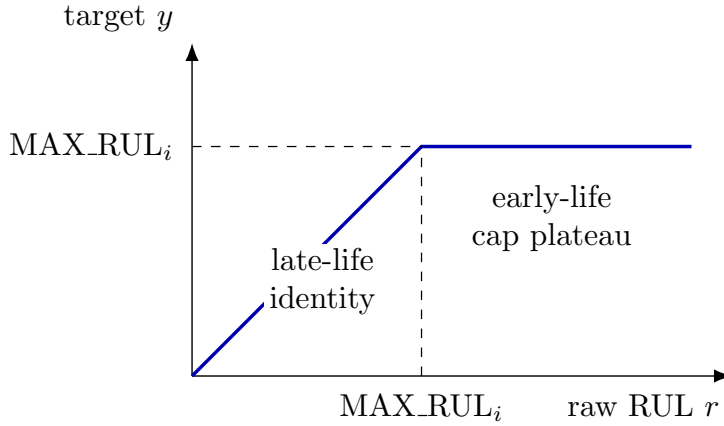


Figure 4.1: RUL target transformation after dataset-wise capping. Late-life RUL values remain unchanged, while earlier windows are mapped to a dataset-specific plateau.

4.3 Windowing and Temporal Downscaling

The main setting uses $T = 256$ native-resolution samples per window. This window size is chosen relative to the cruise-mask method. Because the method requires at least 512 cruise samples, a 256-sample window can fit within a minimum accepted cruise segment while still covering a substantial cruise-stage context.

The benchmark uses window stride as the temporal downscaling method and evaluates sample-rate downsampling in the ablation. Both methods reduce temporal density. The main setting fixes native-resolution windows with stride, while the ablation compares this choice with sample-rate downsampling. The main setting uses stride $s = 10$ at the original N-CMAPSS sample rate. This setting changes how often window starts are selected, but it keeps the native time interval between neighboring samples inside each window. With window size T and stride s , the j -th native-resolution window is

$$X_j = [x_{1+js}, x_{2+js}, \dots, x_{T+js}], \quad (16)$$

where $j = 0, 1, 2, \dots$. Increasing s therefore reduces overlapping, near-duplicate windows while preserving the temporal content of each input sequence.

The difference can be seen from the number of usable windows. For an accepted cruise interval with N native samples, stride-only windowing gives

$$M_{\text{stride}}(N; s) = \begin{cases} \left\lfloor \frac{N - T}{s} \right\rfloor + 1, & N \geq T, \\ 0, & N < T. \end{cases} \quad (17)$$

If the signal is first downsampled in sample rate by a factor k while the T -sample window length is kept unchanged, the reduced sequence contains approximately $\lfloor N/k \rfloor$ samples. The corresponding number of windows is

$$M_{\text{sample}}(N; k) = \begin{cases} \left\lfloor \frac{N}{k} \right\rfloor - T + 1, & \left\lfloor \frac{N}{k} \right\rfloor \geq T, \\ 0, & \left\lfloor \frac{N}{k} \right\rfloor < T. \end{cases} \quad (18)$$

Thus, window stride and sample-rate downsampling produce inputs with the same tensor shape, but the samples inside each input represent different time scales and different temporal detail. With $T = 256$ and $s = k = 10$, a 2560-sample cruise interval yields 231 native-resolution stride windows but only one downsampled window. On the ten-sample length grid shown in Figure 4.2, both counts then increase by one for every additional ten native samples, so the gap remains 230. Accordingly, CruiseBench uses stride as the benchmark parameter for downscaling and keeps the sample-rate factor fixed at one in the main setting.



Figure 4.2: Window-count comparison for fixed window length $T = 256$. The blue dashed line marks the first native-resolution window at $N = 256$, and the yellow dashed line marks the first downsampled window at $N = 2560$. The shaded region marks where the window-count gap between the two settings is constant at 230. Counts are evaluated every ten native samples.

4.4 Evaluation Metrics

The supervised loss is the mean squared error between the prediction $\hat{y}_j$ and the capped RUL target y_j . Reported accuracy uses RMSE in cycles:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{j=1}^N (\hat{y}_j - y_j)^2}. \quad (19)$$

For compatibility with C-MAPSS-style RUL reporting, the asymmetric score proposed by Saxena et al. is adopted [Saxena et al., 2008]. Let

$$\Delta_j = y_j - \hat{y}_j. \quad (20)$$

The score is

$$S_{\text{Saxena}} = \sum_{j=1}^N (\exp(\alpha_j |\Delta_j|) - 1), \quad \alpha_j = \begin{cases} 1/13, & \text{if } \Delta_j > 0, \text{ i.e., RUL is under-estimated,} \\ 1/10, & \text{otherwise.} \end{cases} \quad (21)$$

Lower values are better for both RMSE and the Saxena score. The latter penalizes over-estimation more strongly because delayed maintenance predictions are more risky.

4.5 Main Benchmark Setting

The main benchmark setting is CruiseBench-eta5-W256-S10. It applies the released CPM-N-CMAPSS mask generated by the common-altitude method and follows the task, target, windowing, and metric definitions above. Its fixed configuration is summarized in Table 4.2.

Table 4.2: Summary of the main benchmark setting.

Setting	Value
Cruise-mask artifact	CPM-N-CMAPSS
Cruising-period method	common-altitude
Input feature groups	W, X_s
Excluded feature groups	X_v, θ, A
RUL-cap threshold η	5%
Window size T	256
Stride s	10
Sample-rate factor	1
Evaluation metrics	RMSE, Saxena score

5 Experimental Setup

5.1 Baseline Models

The baseline evaluation uses four standard sequence models: LSTM, GRU, TCN, and TSMixer. These models represent recurrent, convolutional, and MLP-based temporal encoders. All baselines follow the benchmark input and output rules defined in Section 4. The experiments provide reference performance under CruiseBench protocol. Table 5.1 lists the model hyperparameters, which are shared across all datasets in the main setting.

Table 5.1: Baseline model hyperparameters. All models use dropout 0.1, default parameter initialization, and a linear regression head that outputs one scalar RUL value.

Model	Layers/blocks	Hidden or FFN dim.	Kernel/dilation
LSTM	2	32	—
GRU	2	32	—
TCN	2	32	$k = 3$, dilations 1 and 2
TSMixer	2	36	time mixer and series mixer

5.2 Training Setting and Platform

The baseline experiments use a fixed training configuration across models and datasets. Training minimizes mean squared error. Evaluation uses RMSE and the Saxena asymmetric score. Table 5.2 lists the training settings used for the main run grid. The experiments were run on a workstation with an AMD Ryzen Threadripper PRO 5965WX CPU, 130 GB memory, and three NVIDIA GeForce RTX 4090 GPUs.

Table 5.2: Training settings used in the baseline experiments.

Item	Value
Epochs	50
Train batch size	512
Test batch size	512
Optimizer	Adam
Effective weight decay	0
Learning rate	3×10^{-4}
Scheduler	None
Loss	MSE
Repeated runs	Five random seeds
Distributed training	3 GPUs
Mixed precision	No

6 Results and Ablations

6.1 Main Results

The experiment covers the nine accessible subdatasets DS01-005, DS02-006, DS03-012, DS04, DS05, DS06, DS07, DS08a-009, and DS08c-008. DS08d remains excluded because its available HDF5 file could not be read. For each run, the best test RMSE is recorded, and be averaged across the five repeated runs. The main comparison uses CruiseBench-eta5-W256-S10. The $\eta = 0\%$ and $\eta = 10\%$ variants are reported as well.

Table 6.1: Dataset-specific benchmark results under the three RUL-cap settings. RMSE is reported in cycles. Saxena score is reported as $\times 10^4$ and is measured at the epoch with the best test RMSE. Values are mean $\pm$ standard deviation across five repeated runs.

Dataset	Model	$\eta = 0\%$		$\eta = 5\%$		$\eta = 10\%$	
		RMSE	Score	RMSE	Score	RMSE	Score
DS01-005	LSTM	14.61 ± 0.00	21.61 ± 0.07	10.30 ± 1.47	10.29 ± 2.85	9.89 ± 0.10	10.14 ± 0.45
	GRU	6.97 ± 2.41	4.63 ± 2.10	6.69 ± 1.81	4.41 ± 1.59	2.37 ± 1.15	1.14 ± 0.82
	TCN	3.41 ± 0.17	1.82 ± 0.18	2.63 ± 0.25	1.25 ± 0.14	2.20 ± 0.14	1.01 ± 0.10
	TSMixer	2.66 ± 0.15	1.27 ± 0.10	2.25 ± 0.38	1.00 ± 0.21	1.76 ± 0.06	0.74 ± 0.02
DS02-006	LSTM	6.46 ± 5.24	3.53 ± 3.99	6.17 ± 3.45	2.89 ± 2.23	7.20 ± 2.74	3.36 ± 1.86
	GRU	2.83 ± 0.25	0.77 ± 0.11	2.02 ± 0.15	0.51 ± 0.03	1.86 ± 0.52	0.46 ± 0.15
	TCN	3.29 ± 0.24	1.01 ± 0.10	2.52 ± 0.26	0.73 ± 0.10	2.19 ± 0.21	0.60 ± 0.07
	TSMixer	3.17 ± 0.14	0.98 ± 0.08	2.41 ± 0.03	0.65 ± 0.02	2.24 ± 0.16	0.60 ± 0.05
DS03-012	LSTM	12.59 ± 1.96	23.58 ± 6.89	7.46 ± 4.47	11.58 ± 8.88	7.27 ± 3.01	9.72 ± 5.51
	GRU	3.39 ± 0.50	2.72 ± 0.54	2.33 ± 0.11	1.59 ± 0.08	3.08 ± 1.73	2.42 ± 2.03
	TCN	3.42 ± 0.11	2.89 ± 0.16	2.50 ± 0.07	1.93 ± 0.06	2.28 ± 0.09	1.68 ± 0.11
	TSMixer	2.99 ± 0.09	2.33 ± 0.09	2.14 ± 0.03	1.54 ± 0.04	1.94 ± 0.11	1.39 ± 0.09
DS04	LSTM	17.33 ± 0.19	36.78 ± 0.76	13.97 ± 0.03	24.49 ± 0.65	11.92 ± 0.22	18.14 ± 0.76
	GRU	17.28 ± 0.10	37.54 ± 1.15	13.90 ± 0.04	24.54 ± 0.52	11.94 ± 0.08	18.21 ± 0.46
	TCN	9.28 ± 0.42	30.50 ± 37.71	6.98 ± 0.52	6.85 ± 1.09	5.99 ± 0.63	5.47 ± 1.17
	TSMixer	6.90 ± 0.31	7.63 ± 2.56	4.72 ± 0.16	3.51 ± 0.28	4.25 ± 0.22	2.97 ± 0.26
DS05	LSTM	14.93 ± 0.92	20.06 ± 3.11	11.85 ± 0.94	13.25 ± 1.66	10.67 ± 0.07	10.93 ± 0.08
	GRU	8.02 ± 3.49	6.13 ± 3.92	4.44 ± 4.38	3.64 ± 5.54	2.89 ± 0.74	1.44 ± 0.54
	TCN	4.34 ± 0.16	2.47 ± 0.17	3.03 ± 0.12	1.49 ± 0.08	2.33 ± 0.14	1.07 ± 0.09
	TSMixer	3.85 ± 0.32	2.05 ± 0.20	2.60 ± 0.31	1.23 ± 0.18	2.19 ± 0.14	1.05 ± 0.11
DS06	LSTM	16.69 ± 0.67	25.15 ± 2.78	13.69 ± 0.01	17.13 ± 0.18	12.11 ± 0.02	13.84 ± 0.24
	GRU	11.33 ± 5.75	13.11 ± 10.17	7.21 ± 5.51	7.05 ± 7.43	4.86 ± 4.36	3.98 ± 5.50

Table 6.1: Dataset-specific benchmark results under the three RUL-cap settings. RMSE is reported in cycles. Saxena score is reported as $\times 10^4$ and is measured at the epoch with the best test RMSE. Values are mean $\pm$ standard deviation across five repeated runs. (continued)

Dataset	Model	$\eta = 0\%$		$\eta = 5\%$		$\eta = 10\%$	
		RMSE	Score	RMSE	Score	RMSE	Score
DS07	TCN	4.76 ± 0.22	2.80 ± 0.12	3.05 ± 0.16	1.53 ± 0.10	2.59 ± 0.24	1.21 ± 0.12
	TSMixer	4.29 ± 0.37	2.21 ± 0.15	2.61 ± 0.22	1.24 ± 0.14	2.15 ± 0.22	0.99 ± 0.17
	LSTM	15.54 ± 0.04	23.88 ± 0.28	12.17 ± 0.20	15.14 ± 0.69	10.59 ± 0.15	11.84 ± 0.57
	GRU	8.53 ± 2.74	7.01 ± 4.22	8.63 ± 1.53	6.93 ± 2.45	6.54 ± 2.42	5.05 ± 3.65
	TCN	6.39 ± 0.52	4.09 ± 0.58	4.31 ± 0.27	2.32 ± 0.20	4.02 ± 0.41	1.97 ± 0.26
DS08a-009	TSMixer	5.50 ± 0.47	3.13 ± 0.25	4.30 ± 0.25	2.19 ± 0.33	3.42 ± 0.08	1.66 ± 0.08
	LSTM	11.37 ± 2.11	19.79 ± 5.53	10.76 ± 0.89	16.61 ± 1.98	9.54 ± 0.41	13.05 ± 1.28
	GRU	9.41 ± 2.36	14.89 ± 5.77	9.26 ± 2.15	13.38 ± 4.18	7.32 ± 1.05	8.92 ± 1.99
	TCN	7.92 ± 0.25	12.78 ± 1.17	6.92 ± 0.13	9.00 ± 0.38	6.02 ± 0.05	6.71 ± 0.13
DS08c-008	TSMixer	8.29 ± 0.38	13.60 ± 2.13	7.32 ± 0.42	10.12 ± 1.27	6.43 ± 0.28	7.61 ± 0.59
	LSTM	10.56 ± 1.02	7.66 ± 1.33	8.47 ± 0.73	5.27 ± 0.86	7.70 ± 0.39	4.64 ± 0.36
	GRU	5.68 ± 3.34	3.19 ± 2.90	5.68 ± 2.37	2.99 ± 1.91	4.25 ± 2.62	2.06 ± 1.86
	TCN	3.85 ± 0.15	1.61 ± 0.09	3.12 ± 0.06	1.21 ± 0.03	2.66 ± 0.11	0.98 ± 0.06
	TSMixer	3.37 ± 0.13	1.33 ± 0.07	2.76 ± 0.17	0.98 ± 0.09	2.43 ± 0.07	0.81 ± 0.02

Table 6.1 shows that the cruise-only setting retains learnable RUL structure for all baseline families, while difficulty remains dataset-dependent. Averaging the dataset-specific entries under the main $\eta = 5\%$ setting, TSMixer obtains the lowest mean RMSE, 3.46 ± 1.71 , and the lowest Saxena score, $(2.50 \pm 2.99) \times 10^4$. TCN follows with 3.89 ± 1.81 RMSE and $(2.92 \pm 2.92) \times 10^4$ score. This supports the benchmark argument. Ordinary sequence models can produce meaningful predictions in the fixed cruise-stage regime, but model choice still changes accuracy and stability under the same data setting.

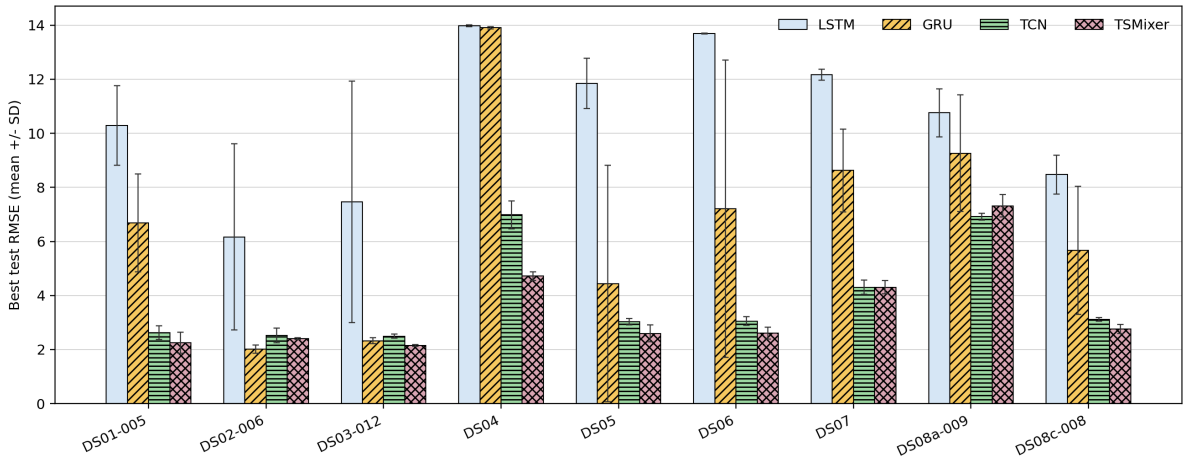


Figure 6.1: Best test RMSE by dataset under the main $\eta = 5\%$ setting. Bars show means and error bars show standard deviation across five repeated runs.

Figure 6.1 highlights DS04 and DS08a-009 as more difficult cases in this benchmark setting. This pattern is consistent with the benchmark design. Once cruise rows are isolated and leakage-prone metadata are removed, dataset-specific degradation and operating-history differences still matter. CPM-N-CMAPSS preserves this difference rather than make all datasets equally easy.

6.2 Ablation Studies

6.2.1 Flight-stage sensitivity

This ablation examines whether model ranking and repeat-to-repeat stability change when the flight-stage constraint is removed. The cruise-stage family uses the released CPM-N-CMAPSS mask and samples windows only from accepted cruising intervals. The no-mask family keeps the same feature rule, target construction, and model settings, but samples windows from complete flight cycles that may include climb, cruise, descent, and transition segments. Because the two families contain different window populations, the comparison is interpreted as a flight-stage sensitivity analysis rather than a direct leaderboard comparison.

Figure 6.2 compares TCN and TSMixer, the two strongest baselines shown in Table 6.1. Under the cruise-stage mask, TSMixer has the lower average RMSE, 3.46 versus 3.89 for TCN. In the no-mask family, the ordering reverses slightly. TCN averages 2.95 and TSMixer averages 3.00.

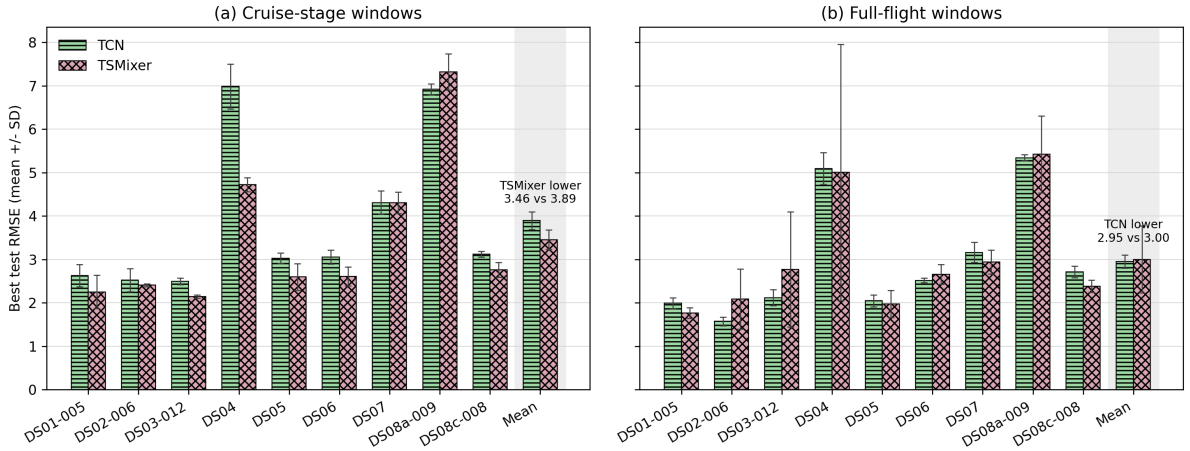


Figure 6.2: TCN and TSMixer ranking under (a) cruise-stage and (b) no-mask full-flight experiment families. Dataset bars show mean best test RMSE with standard deviation across five repeated runs. The shaded Mean group shows the mean of subdataset RMSE means, with the error bar equal to the average repeat-to-repeat standard deviation across subdatasets.

The ranking reversal indicates that the no-mask setting changes the model comparison by adding operating-regime variation to the prediction windows. Under the full-flight window population, TCN gains more than TSMixer from the added climb, cruise, descent, and transition patterns. This supports the role of CruiseBench as a controlled construction because fixing the flight stage separates cruise-stage RUL prediction from performance gains associated with mixed-regime inputs.

Table 6.2 further shows that the effect is architecture-dependent. TSMixer has a clear increase in repeat-to-repeat deviation when moving from cruise-stage windows to full-flight windows. Its average repeat standard deviation rises from 0.22 to 0.77, with the largest increases on DS03-012, DS04, and DS08a-009. TCN does not show the same instability pattern, with repeat standard deviation changing from 0.21 under cruise-stage windows to 0.16 under full-flight windows. This suggests that TSMixer is more sensitive to mixed operating regimes under the current setup, and that stronger regime-aware modeling may be needed when it is applied to full-flight inputs.

Table 6.2: Subdataset-specific repeat-to-repeat RMSE deviation for TCN and TSMixer under cruise-stage and no-mask full-flight windows. Values are standard deviations across five repeated runs. Δ SD is full-flight minus cruise-stage repeat SD for the same model and subdataset.

Dataset	TCN cruise SD	TCN full SD	TCN Δ SD	TSMixer cruise SD	TSMixer full SD	TSMixer Δ SD
DS01-005	0.25	0.13	-0.13	0.38	0.12	-0.26
DS02-006	0.26	0.10	-0.17	0.03	0.69	+0.66
DS03-012	0.07	0.19	+0.11	0.03	1.33	+1.30
DS04	0.52	0.37	-0.15	0.16	2.95	+2.79
DS05	0.12	0.14	+0.01	0.31	0.31	+0.00
DS06	0.16	0.05	-0.11	0.22	0.23	+0.00
DS07	0.27	0.23	-0.04	0.25	0.28	+0.03
DS08a-009	0.13	0.07	-0.05	0.42	0.89	+0.47
DS08c-008	0.06	0.13	+0.07	0.17	0.14	-0.03
Mean	0.21	0.16	-0.05	0.22	0.77	+0.55

The window counts show that this diagnostic changes both flight-stage composition and window population size. The cruise-stage setting uses 1,101,166 train windows and 614,293 test windows, while the no-mask full-flight setting uses 4,312,736 train windows and 2,490,297 test windows. The assessment therefore shows how model ranking and stability change when the benchmark constraint is removed, rather than a direct result comparison. It shows that model ranking and training stability can change when flight-stage control is removed, which motivates reporting cruise-stage results separately from full-flight results.

6.2.2 Window stride versus sample-rate downsampling

Table 6.3 compares two temporal downscaling methods for TSMixer under the cruise-stage mask. The sample-rate setting downsamples by a factor of 10 with stride 1, while the window-stride setting keeps native-resolution windows with dataset-specific train/test strides. The custom-stride setting uses fewer training windows overall, 353,753 versus 360,938, and more test windows, 178,058 versus 176,425. Even under that slightly less favorable train/test count balance, it improves the average TSMixer RMSE from 3.85 to 3.69 and is better on DS03-012, DS04, DS05, DS06, and DS08c-008.

Table 6.3: TSMixer comparison between sample-rate downsampling and native-resolution custom stride. Window counts are train/test. RMSE values are mean $\pm$ standard deviation across five repeated runs. The final row reports total windows and mean $\pm$ standard deviation across dataset-level RMSE means. Negative Δ favors custom stride.

Dataset	Sample10 windows	Sample10 RMSE	Custom stride	Custom windows	Custom RMSE	Δ
DS01-005	33,959/16,106	2.03 $\pm$ 0.29	35/40	33,496/16,124	2.16 $\pm$ 0.16	+0.13
DS02-006	76,557/9,416	2.94 $\pm$ 0.14	23/38	74,094/9,535	3.23 $\pm$ 0.15	+0.29
DS03-012	32,674/33,901	2.75 $\pm$ 0.14	40/30	32,141/33,951	2.20 $\pm$ 0.14	-0.55
DS04	53,354/22,529	9.54 $\pm$ 2.24	29/36	51,597/22,667	8.02 $\pm$ 1.61	-1.52
DS05	31,204/16,870	2.73 $\pm$ 0.23	35/36	30,946/17,254	2.48 $\pm$ 0.34	-0.25
DS06	30,981/16,870	2.73 $\pm$ 0.32	35/36	30,275/17,103	2.59 $\pm$ 0.47	-0.14
DS07	32,090/19,277	2.84 $\pm$ 0.37	34/35	31,529/19,533	3.21 $\pm$ 0.25	+0.37
DS08a-009	21,343/33,379	6.14 $\pm$ 0.43	51/28	21,131/33,785	6.63 $\pm$ 0.32	+0.49
DS08c-008	48,776/8,077	2.94 $\pm$ 0.41	23/58	48,544/8,106	2.69 $\pm$ 0.25	-0.25
Mean/total	360,938/176,425	3.85 $\pm$ 2.43	-	353,753/178,058	3.69 $\pm$ 2.12	-0.16

The result supports window stride as the benchmark’s downscaling method. Window stride provides a finer adjustment range than sample-rate downsampling. It can change the number of extracted windows without changing the native temporal spacing inside each 256-sample input. Sample-rate downsampling instead changes the physical time span represented by a fixed-length input and offers a coarser control over usable window counts. The custom-stride setting further shows this flexibility, because train and test splits can use different stride values to meet a target data budget.

6.2.3 RUL-cap sensitivity

The $\eta = 10\%$ and $\eta = 0\%$ settings evaluate sensitivity to the RUL-cap threshold. They all use CPM-N-CMAPSS feature rule, and windowing rule as the main $\eta = 5\%$ setting, but apply different dataset-wise caps for RUL target construction. The resulting RMSE values change because both the target scale and the regression target are modified. These results therefore characterize benchmark-setting sensitivity rather than intrinsic dataset difficulty.

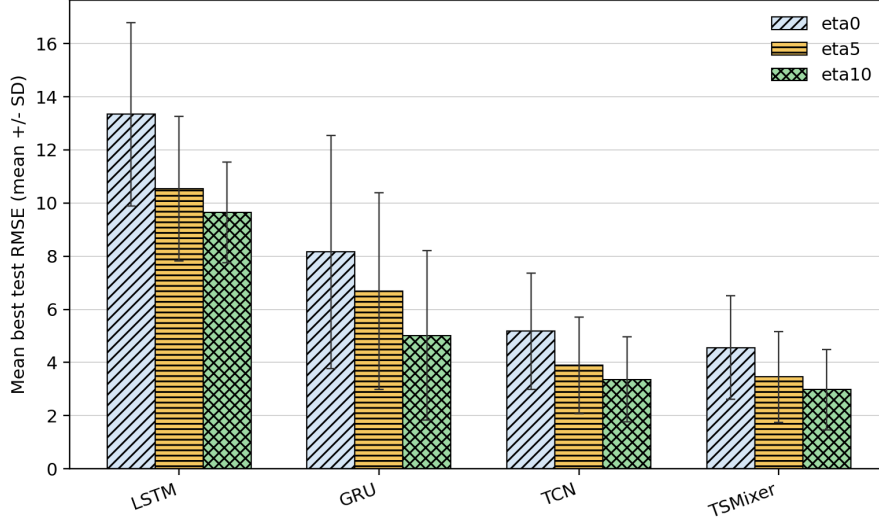


Figure 6.3: Average RMSE sensitivity to the RUL-cap threshold. Bars show means across the nine accessible subdatasets and error bars show standard deviation across dataset-level means.

Table 6.1 and Figure 6.3 show that cap choice has a visible effect on RMSE and score. The $\eta = 10\%$ setting gives smaller absolute RMSE because it uses a stricter cap, while $\eta = 0\%$ gives larger values because it uses the full minimum abnormal length as the cap. This supports reporting the cap as part of the setting name. Results obtained with different cap rules should not be merged into a single comparison.

Together, the ablation studies support three findings. First, flight-stage selection affects model ranking and stability, which motivates reporting the cruise-stage setting separately from no-mask full-flight results. Second, window stride provides a practical temporal downscaling choice because it adjusts window density while preserving native-resolution within-window dynamics. Third, the RUL-cap threshold changes reported RMSE and should be treated as part of the benchmark setting.

6.3 Learning dynamics

The learning curves provide a useful check beyond final numbers. Representative train/test trajectories are shown in Figure 6.4. The curves show that the reported minima are reached through stable optimization and are not isolated scalar values. They also indicate that simple sequence baselines can fit the cruise-only windows without requiring flight-class or health-state inputs.

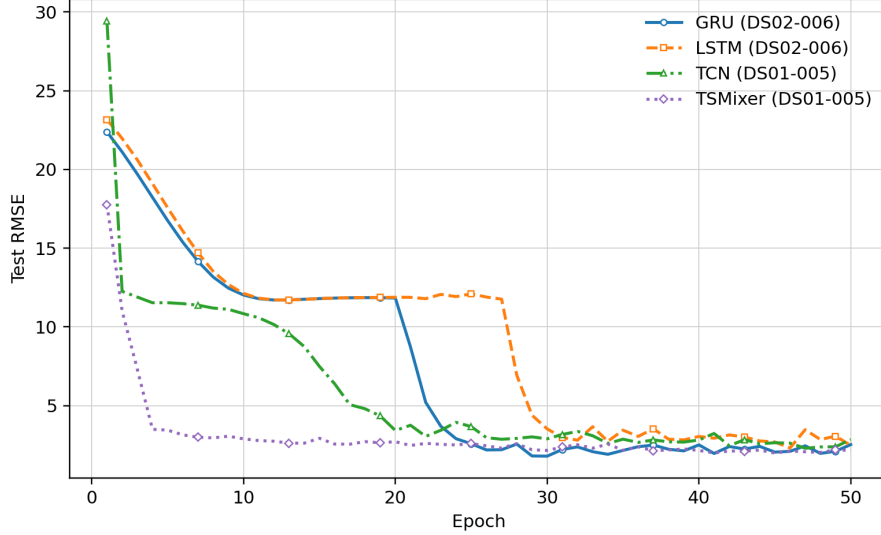


Figure 6.4: Representative $\eta = 5\%$ learning curves. The plotted runs show the relation between optimization progress and the best test RMSE used in Table 6.1.

7 Discussion

CruiseBench isolates a stable flight stage and therefore reduces flight-stage composition as a confounding factor in model comparison. The results shown in Table 6.1 support this framing. Several ordinary sequence encoders reach low RMSE without using health state, flight class, unit id, cycle id, or sample index as predictive features. This makes the benchmark useful as a controlled sub-benchmark, but it also narrows the operational question. Climb and descent may contain useful degradation information that is intentionally removed here. Full-flight N-CMAPSS evaluation remains necessary for models intended to operate across the entire flight envelope.

The RUL-cap policy is also a setting choice rather than a directly observed detectability threshold. The exponential abnormal-degradation law motivates a smaller cap than the full abnormal period, but η should be treated as a benchmark setting. The $\eta = 0\%$, $\eta = 5\%$, and $\eta = 10\%$ results differ enough that they should not be combined into one leaderboard. Reporting each η setting separately makes this choice clear.

Finally, the cruise mask is fixed given the source HDF5 files and method settings. This supports reproducibility, but the extraction method still contains assumptions. Cruise is represented as a long high-altitude common-level plateau, and the final accepted interval is selected by duration and altitude stability. These assumptions are appropriate for a cruise-stage benchmark and should not be generalized to every flight stage.

8 Conclusion

This paper presented CruiseBench, a cruise-stage RUL benchmark derived from N-CMAPSS, and CPM-N-CMAPSS, a cruise-stage mask artifact that stores cycle-local cruising intervals for the nine accessible subdatasets. The benchmark applies a fixed protocol to masked cruise-stage rows. Scenario descriptors and measured sensors are used as inputs, virtual sensors, health parameters, and auxiliary metadata are excluded from the feature tensor, windows preserve the native time resolution, and RUL labels use dataset-wise caps. Baseline experiments with LSTM, GRU, TCN, and TSMixer show that the cruise-stage setting retains learnable RUL structure,

with TSMixer obtaining the lowest average RMSE and Saxena score under CruiseBench-eta5-W256-S10.

The ablation studies further show that flight-stage selection, temporal downscaling method, and RUL-cap threshold affect reported results and should be treated as explicit benchmark settings. With its fixed cruise-stage protocol, CruiseBench supports controlled RUL model comparison. CPM-N-CMAPSS also provides a stage-specific data foundation for future transfer-learning and domain-adaptation studies. Future work can extend the same mask-based design to additional operating regimes, evaluate cross-stage transfer, and test specialized prognostics architectures under fixed data and target-construction rules.

9 Reproducibility

The open-source package provides the released cruise mask and all tools needed to regenerate and inspect it. The raw N-CMAPSS dataset files are not redistributed. Users place the official HDF5 files in the expected local data directory, then either use the released mask directly or regenerate it with the provided exporter.

The released mask file is:

```
dataset/cruise_mask/common_altitude_cruise_mask.h5
```

It contains 7473 cycle rows and 7053 accepted cruise cycles. The relevant HDF5 attributes are:

Attribute	Value
profile	<code>common_altitude</code>
method	<code>common_altitude</code>
created at	2026-04-27T18:44:21
interval unit	<code>cycle_local_sample_index</code>
interval end	<code>inclusive</code>

For each experiment, reports should include the source dataset list, cruise-mask settings, window size, downscaling method, feature set, dataset-wise RUL caps and model hyperparameters.

Acknowledgement

This research was partially supported by the National Natural Science Foundation of China (No. 52075349) and Sichuan Science and Technology Program under Grant (No. 2025YFHZ0157).

References

- Manuel Arias Chao, Chetan Kulkarni, Kai Goebel, and Olga Fink. Aircraft engine run-to-failure dataset under real flight conditions for prognostics and diagnostics. *Data*, 6(1):5, 2021. doi: 10.3390/data6010005. URL <https://www.mdpi.com/2306-5729/6/1/5>.
- Manuel Arias Chao, Chetan Kulkarni, Kai Goebel, and Olga Fink. Fusing physics-based and deep learning models for prognostics. *Reliability Engineering and System Safety*, 217:107961, 2022. doi: 10.1016/j.ress.2021.107961.
- Leonardo Lucio Custode, Hyunho Mo, Andrea Ferigo, and Giovanni Iacca. Evolutionary optimization of spiking neural P systems for remaining useful life prediction. *Algorithms*, 15(3): 98, 2022. doi: 10.3390/a15030098. URL <https://www.mdpi.com/1999-4893/15/3/98>.

- Andre Listou Ellefsen, Sergey Ushakov, Vilmar Æsøy, and Houxiang Zhang. Validation of data-driven labeling approaches using a novel deep network structure for remaining useful life predictions. *IEEE Access*, 7:71563–71575, 2019. doi: 10.1109/ACCESS.2019.2920297.
- Florent Forest, Katharina Rombach, and Olga Fink. Interpretable prognostics with concept bottleneck models. *Information Fusion*, 124:103427, 2025. doi: 10.1016/j.inffus.2025.103427.
- Zhan Gao, Yumeng Lei, Jun Wu, Weixiong Jiang, Yuanhang Wang, and Xiyuan Ye. VSC-Net: Versatile spatiotemporal convolution network with multi-sensor signals for remaining useful life prediction of mechanical systems. *Information Fusion*, 123:103288, 2025. doi: 10.1016/j.inffus.2025.103288.
- Felix O. Heimes. Recurrent neural networks for remaining useful life estimation. In *2008 International Conference on Prognostics and Health Management*, pages 1–6, 2008. doi: 10.1109/PHM.2008.4711422.
- Mingxing Huang, Lanying Yang, Gang Jiang, Xingan Hao, Hong Lu, Hang Luo, Peng Wang, and Jinyang Li. Resconv-xlstm: An improved xlstm model with spatiotemporal feature extraction capability for remaining useful life prediction of aero-engine. *Results in Engineering*, 26: 105513, 2025. doi: 10.1016/j.rineng.2025.105513.
- Diogo Landau, Ingeborg de Pater, Mihaela Mitici, and Nishant Saurabh. Federated learning framework for collaborative remaining useful life prognostics: An aircraft engine case study. *Future Generation Computer Systems*, 174:107945, 2026. doi: 10.1016/j.future.2025.107945.
- Xiang Li, Qian Ding, and Jian-Qiao Sun. Remaining useful life estimation in prognostics using deep convolution neural networks. *Reliability Engineering and System Safety*, 172:1–11, 2018. doi: 10.1016/j.ress.2017.11.021.
- Zhaofeng Liu, Xiaoqing Zheng, Anke Xue, and Ming Ge. Multi-scale temporal-spatial feature-based hybrid deep neural network for remaining useful life prediction of aero-engine. *ACS Omega*, 9(48):47410–47427, 2024. doi: 10.1021/acsomega.4c03873.
- Mingyuan Qin, Gangyan Xu, Xi Chen, and Wei Li. Multi-sensor spatial and multiscale temporal transformer guided by frequency-domain attention for aeroengine remaining useful life prediction. *Reliability Engineering and System Safety*, 272:112535, 2026. doi: 10.1016/j.ress.2026.112535.
- Yonglei Ren, Zong Meng, and Jimeng Li. Spatial-temporal graph hybrid neural network for remaining useful life prediction of aero-engines. *Reliability Engineering and System Safety*, 274:112433, 2026. doi: 10.1016/j.ress.2026.112433.
- Abhinav Saxena, Kai Goebel, Don Simon, and Neil Eklund. Damage propagation modeling for aircraft engine run-to-failure simulation. In *2008 International Conference on Prognostics and Health Management*, pages 1–9, 2008. doi: 10.1109/PHM.2008.4711414. URL <https://ieeexplore.ieee.org/document/4711414>.
- Ya Song, Laurens Blik, Yaoxin Wu, and Yingqian Zhang. Enhancing remaining useful life prediction with ensemble multi-term fourier graph neural networks. *Transactions on Machine Learning Research*, 2025(01), 2025. URL <https://openreview.net/pdf?id=tzFjcVqmxw>.
- Peiwen Wang, Shaozhang Niu, Haoliang Cui, and Wen Zhang. Gpt-based equipment remaining useful life prediction. In *Proceedings of the ACM Turing Award Celebration Conference-China 2024*, pages 159–164, 2024.

- Chenchen Wu, Jialong He, Liang Wang, Chi Ma, and Yan Liu. A dual-path architecture based on time series decomposition and degradation correction for remaining useful life prediction of aero-engine. *Computers and Industrial Engineering*, 203:110964, 2025. doi: 10.1016/j.cie.2025.110964.
- Tiantian Xu, Guangjie Han, Hongbo Zhu, Chuan Lin, and Jinlin Peng. Multiscale bls-based lightweight prediction model for remaining useful life of aero-engine. *IEEE Transactions on Reliability*, 73(4):1757–1767, 2024. doi: 10.1109/TR.2023.3349201.
- Junqi Zeng and Zhenglin Liang. A deep gaussian process approach for predictive maintenance. *IEEE Transactions on Reliability*, 72(3):916–933, 2023. doi: 10.1109/TR.2022.3199924.
- Zhizheng Zhang, Wen Song, Qiong Wu, Wenxu Sun, Qiqiang Li, and Lei Jia. A novel local enhanced channel self-attention based on transformer for industrial remaining useful life prediction. *Engineering Applications of Artificial Intelligence*, 141:109815, 2025. doi: 10.1016/j.engappai.2024.109815.
- Zhihao Zhou, Yiran Shao, Bo Jiang, Peng Yao, Jinfu Liu, and Daren Yu. An aero-engine remaining useful life prediction method based on deformable convolutional residual attention enhanced kolmogorov-arnold networks with prediction advance and shape constraints. *Reliability Engineering and System Safety*, 268:111977, 2026. doi: 10.1016/j.res.2025.111977.
- Jiangyan Zhu, Jun Ma, Jiande Wu, and Lekang Fan. Efficient temporal adjacent transformer for remaining useful life prognostics. *Engineering Applications of Artificial Intelligence*, 166:113572, 2026. doi: 10.1016/j.engappai.2025.113572.